1) Block header fields

A Bitcoin block header is a compact 80-byte summary of the block and contains six fields: version, previous block hash, Merkle root, timestamp, difficulty target, and nonce. These fields let the block identify its rule version, link to its parent, summarize its transactions, record approximate creation time, define mining difficulty, and give miners a value to change while searching for a valid hash. So the header is like the essential metadata and identity of the block.

---

2) Block header hash

The block header hash is the hash of the block header, computed in Bitcoin using double SHA-256. This is the value miners try to make small enough to satisfy the current difficulty target. The hash proves the miner did the required work and also acts as the block's identifier in the chain. So block mining is really about finding a valid block header hash, not hashing the whole transaction list again and again directly.

---

3) Why Bitcoin struggles for coffee / micropayments

Bitcoin struggles for coffee-sized micropayments because the base layer is too slow, too public, and often too expensive for very small purchases. If every cup of coffee had to wait for an on-chain confirmation, users would face long delays and unnecessary fees. Also, putting every tiny everyday purchase on a public ledger is not ideal for privacy. So the problem is not that Bitcoin cannot represent small value, but that on-chain usage is heavy for tiny frequent payments.

---

4) Why TPS and 10-minute block time matter

Bitcoin handles only a small number of transactions per second, and blocks are generated on average every 10 minutes. This means the network cannot process large numbers of instant everyday payments the way a high-speed payment network can. For retail use, people usually want fast feedback and high throughput, but Bitcoin's base layer is designed more for security and decentralization than for rapid mass micropayment processing. That is why TPS and block time are major practical constraints.

---

5) What Bitcoin Script can do

Bitcoin Script can do much more than just check one signature. It can verify ownership, combine conditions, require multiple approvals, lock funds until a future time, or delay spending for a period after confirmation. This means Bitcoin transactions can contain real spending logic, not just simple transfers. So Bitcoin Script is the rule engine that lets outputs say exactly under what conditions they may be spent.

---

6) Miniscript

Miniscript is a more structured and human-readable way to express Bitcoin spending policies, which can then be compiled into raw Bitcoin Script. It is useful because raw Bitcoin Script is stack-based, low-level, and easy to make mistakes in. Miniscript helps developers reason about complex conditions like multisig or time locks without manually writing difficult opcode sequences. So it makes Bitcoin scripting safer and easier to understand.

---

7) UTXO rules recap

In the UTXO model, every Bitcoin transaction consumes old unspent outputs and creates new outputs for future use. Each input must reference exactly one previous output, and each output must define both a value and a locking script. The total input value must be at least as large as the total output value. So Bitcoin does not use account balances directly; it uses spendable pieces of value called UTXOs.

---

8) Spending an output

To spend an output, a new child transaction must point to that previous output and provide the correct unlocking data, such as a signature. The old output contains a locking condition in its scriptPubKey, and the new input provides the matching unlocking script or witness. If the combined script evaluation succeeds, the old output is fully spent and replaced by new outputs in the child transaction. So spending is really the process of proving the right to unlock an earlier UTXO.

---

9) Parent transaction vs child transaction

The parent transaction is the earlier transaction that created an output, and the child transaction is the later transaction that tries to spend that output. The parent is also called the funding transaction because it provides the locked value, while the child is the spending transaction because it consumes that value. This parent-child relationship is central to the UTXO model. So a child transaction always depends on something created by a parent transaction.

---

10) Transaction chaining

Transaction chaining means Bitcoin transactions form a spending chain over time: one transaction creates outputs, later transactions spend those outputs, and then create new outputs again. So the flow of value can be traced across many linked transactions, not just across blocks. This is why Bitcoin history can be seen as both a blockchain and a chain of spending relationships. So transactions themselves form a graph or chain of value movement.

---

11) UTXO database / chainstate

Although UTXOs are created and spent across many blocks, nodes maintain a separate UTXO database, often called chainstate, to keep track of all currently unspent outputs. This makes transaction validation much faster because the node does not have to scan the entire blockchain every time it checks a new transaction. The database stores only the outputs that are still spendable at the current moment. So chainstate is the live spendable-state view of Bitcoin.

---

12) Simplified validation algorithm

To validate a new Bitcoin transaction, a node first checks that every referenced parent UTXO exists in the UTXO database and has not already been spent. Then it checks whether the unlocking data matches the spending conditions of the parent output. After that, it ensures the sum of input values is not less than the sum of output values. If everything is valid, the old parent UTXOs are removed from the database and the new child outputs are added.

---

13) Different output indices can both be valid

Two child transactions can both be valid if they reference different output indices of the same parent transaction. This is because a single parent transaction may create multiple separate outputs, and each of those outputs is its own independent UTXO. So even if the parent txid is the same, spending output index 1 is different from spending output index 2. The key lesson is that txid alone is not enough — the index matters too.

---

14) Race between transactions

A race between transactions happens when two child transactions try to spend the same UTXO. Since a UTXO can only be spent once, the network can accept only one of those competing transactions into the confirmed chain. The one that gets confirmed first wins, and the other becomes invalid as a double-spend attempt. So a transaction race is really a competition over who gets to consume a particular UTXO first.

---

15) Double spending as competition for same UTXO

In the UTXO model, double spending happens when two different transactions both try to spend the exact same unspent output. Since only one can succeed, they are directly in conflict with each other. The accepted chain will confirm one and reject the other. So in Bitcoin, double spending is not an abstract idea — it appears very concretely as two transactions competing for one UTXO.

---

16) Valid but wasteful transaction

A Bitcoin transaction can be technically valid yet still be a very bad idea economically. For example, if Alice spends a 1.5 BTC UTXO to send Bob only 0.5 BTC and forgets to make a change output, then the remaining 1 BTC becomes transaction fee for the miner. The network still accepts it because inputs are greater than outputs, but Alice loses far too much. So “valid” in Bitcoin does not always mean “smart.”

---

17) Change output

A change output is the extra output that sends the leftover value back to the sender when a spent UTXO is larger than the intended payment amount. Because Bitcoin consumes old

outputs whole, any unused remainder must be explicitly returned to the sender in a new output. This is similar to receiving change when you pay with a larger cash note. So a standard Bitcoin payment usually has one output for the recipient and one change output back to the sender.

---

18) Collaborative transaction

A collaborative transaction is a transaction whose full set of inputs and outputs is agreed upon by multiple parties before broadcast. One party may create the unsigned transaction, then each participant signs it with their own private key. The signatures are exchanged until someone holds the fully signed version. This is useful for multisig, channels, and other shared control situations where no single person can complete the transaction alone.

---

19) PSBT

PSBT stands for Partially Signed Bitcoin Transaction. It is a transaction package that has some required signatures but not all of them yet, so it cannot be broadcast as final to the network. PSBTs are useful when multiple people or devices must cooperate to sign the same transaction. So a PSBT is basically an in-progress Bitcoin transaction waiting for the remaining signatures.

---

20) Satoshis as micropayment unit

A satoshi, or sat, is the smallest unit of Bitcoin, and 1 BTC equals 100 million sats. Thinking in sats makes Bitcoin feel more usable for small payments, because a coffee can be described as maybe 1000 sats instead of a tiny awkward decimal fraction of BTC. This changes the mental framing from “Bitcoin is too expensive” to “Bitcoin has a small usable unit.” So sats are the practical everyday denomination for micropayment thinking.

---

21) Fee depends on transaction size, not value

In Bitcoin, transaction fees are based mainly on the size of the transaction in bytes, not on the dollar value being transferred. That means a $1 payment can pay the same fee as a $100,000 transfer if both transactions take up similar space in the block. This is why micropayments are difficult on-chain: small-value payments do not automatically get small-value fees. So block space, not payment amount, is what really drives fee cost.

---

22) Funding transaction

A funding transaction is the on-chain transaction used to open a payment channel by locking funds into a shared output, often a 2-of-2 multisig output. This transaction is the base that later off-chain balance updates depend on. It is called the funding transaction because it provides the actual locked value that the channel will redistribute over time. So the channel starts when the funding transaction is safely confirmed on-chain.

---

23) Commitment transaction

A commitment transaction is a transaction that represents the current agreed balance split inside a payment channel. It is usually kept off-chain during normal channel use and gets updated every time the participants change their balance state. If needed, the latest commitment transaction can be published on-chain to settle the channel. So the commitment transaction is the off-chain record of who currently owns what within the channel.

---

24) Unobserved off-chain events

Unobserved off-chain events are the balance updates inside a payment channel that never get written to the blockchain individually. For example, if Alice buys coffee several times over several days, the balance may be updated after each purchase off-chain, but the public blockchain only sees the channel opening and final settlement. This improves privacy and reduces blockchain load. So many real payments can happen without each one becoming a public on-chain event.

---

25) Payment channel definition

A payment channel is a setup that allows two parties to make repeated payments between each other without using the blockchain for every single transaction. The blockchain is mainly used at the start to open the channel and at the end to close it, while the internal balance updates happen off-chain. This makes payments faster, cheaper, and more private. So a payment channel is like a private temporary payment lane anchored to blockchain security.

---

26) Payment channel step-by-step flow

In a payment channel, Alice and Bob first agree to open the channel and create a funding transaction, usually with shared control such as 2-of-2 multisig. Before broadcasting that funding transaction, the payer gets a pre-signed commitment transaction to ensure the funds can be recovered safely. Then the funding transaction is published on-chain. After that, Alice and Bob update their balances off-chain by creating newer commitment transactions whenever payments happen. Finally, when they want to settle, the latest commitment transaction is broadcast and the channel closes on-chain.